[Fase 1]Proyecto Final

Integrantes:

- Carlos Andrés Rodríguez Pelayo
- Néstor Andrés Rivera Quintana
- Gerardo Josué Morales Sánchez
- Jared Vera Jaime

INTRODUCCION

Procesador MIPS

El procesador MIPS es un referente de la arquitectura RISC. Sus características generales es que todas sus instrucciones son de 32 bits sin excepción, lo que genera un flujo continuo para cualquier proceso realizado.

El repertorio general de instrucciones de MIPS se divide en tres formatos principales, que son: Tipo R (registro). Utilizadas para operaciones aritméticas. Tipo I (inmediato). Tipo J (Salto).

Para fines específicos de este proyecto, se utilizaran las instrucciones de tipo R. Que tienen una funcionalidad especial, descrita a continuación.

Funcionalidad:

Los 32 bits anteriormente mencionados entran separados en distintos campos definidos, que siguen a otros módulos para lograr la operación deseada. Estos campos son:

OpCode, En instrucciones type R siempre es 6b’0

rs. Registro del primer operando

rt. Registro del segundo operando

rd, Registro destino, obtiene el resultado.

Shamt. Usado para instrucciones de desplazamiento. Siempre 5b’0 en Instrucciones typeR

Funct. El módulo de control de la ALU ha sido diseñado para decodificar distintas operaciones a través de su código único de operación o código de función (funct).

Que algoritmos se pueden hacer con las instrucciones provistas

El algoritmo elegido por nosotros para implementar es el BubbleSort, dicho algoritmo necesita principalmente de tres instrucciones clave.

Comparación. Se usa SLT (Set Less Than), compara dos registros y define cual es el mayor

Intercambio. En ensamblador, mover un número a la derecha, en el caso preciso del algoritmo es el más grande, no depende de una sola instrucción, sino de un par. Se usa Lw (Load Word), que guarda el registro para trabajar con el posteriormente leyendo ambos registros, y posteriormente Sw (store Word), para intercambiarlos de posición, (sobrescribir la memoria cambiándolos de posición)

Ciclo/Bucle. Para realizar un ciclo for en ensamblador y permitir hacer las pasadas necesarias para comparación de registros, debe hacerse de forma manual, utilizando entre otras cosas, las instrucciones: beq (branch on equal), bne (branch on not equal), j (jump).

DESARROLLO

Desarrollo de los modulos

Módulo PC

Para llevar a cabo el modulo pc, nos basamos en el diagrama observando que cuenta con 3 entradas y 1 salida, siendo una entrada el reloj que dará el pulso de cada buffer, este módulo se encarga de recuperar la instrucción actual y la ubicación del siguiente, además de contar con una entrada de reset.

Módulo add32

Para la implementación del módulo add32, nos basamos en el diagrama, el cual contiene 2 entradas y una salida. Este módulo nos ayuda a obtener la instrucción dentro de nuestra memoria de datos, donde dicho modulo recupera 4 bytes de todas las instrucciones.

Módulo IMEM (Memoria de Instrucciones):

Para la implementación del módulo IMEM, nos basamos en el diagrama observando que cuenta con 1 entrada y 1 salida. Este módulo se encarga de almacenar el programa completo a ejecutar y nos ayuda a extraer el código binario de 32 bits de la instrucción correspondiente, basándose directamente en la dirección física proporcionada por el PC.

Módulo UC (Unidad de Control):

Para llevar a cabo el módulo UC, observamos que el diagrama contiene 1 entrada principal (el código de operación) y 5 salidas lógicas de control (las cuales tenían que ser registros). Este módulo se encarga de leer los primeros 6 bits de la instrucción actual y determinar qué banderas se deben encender o apagar para configurar correctamente el resto de los componentes durante ese ciclo.

Módulo BR (Banco de Registros):

Para la implementación del módulo BR, nos basamos en el diagrama observando que cuenta con 6 entradas (incluyendo el reloj y la señal de permiso de escritura) y 2 salidas. Este módulo nos ayuda a leer los operandos guardados en los registros especificados por la instrucción, y también se encarga de escribir de forma síncrona los resultados finales de vuelta en un registro destino designado.

Módulo AluControl (Control de la ALU):

Para llevar a cabo el módulo AluControl, determinamos que contiene 2 entradas y 1 salida. Este módulo actúa como un traductor auxiliar. Nos ayuda a decodificar el campo "funct" de la instrucción junto con la señal enviada por la Unidad de Control, encargándose de generar el código exacto de 4 bits que le dirá a la ALU qué operación matemática realizar.

Módulo ALU (Unidad Aritmético Lógica):

Para la implementación del módulo ALU, nos basamos en el diagrama observando que cuenta con 3 entradas (dos operandos y una señal que selecciona la operación a realizar) y 1 salida principal. Este módulo se encarga de ejecutar el procesamiento de datos pesado, realizando todas las operaciones matemáticas y lógicas (como sumas, restas o compuertas AND/OR) entre los datos recuperados, entregando el resultado de dicha operación.

Módulo MEM (Memoria de Datos):

Para llevar a cabo el módulo MEM, nos percatamos que contiene 5 entradas (incluyendo el reloj y los permisos de lectura/escritura) y 1 salida. Este módulo nos ayuda a simular la memoria RAM de nuestra arquitectura, encargándose de guardar permanentemente el resultado de un registro en una dirección de memoria, o de recuperar un dato almacenado de esa misma dirección para enviarlo de vuelta al procesador.

Módulo MUX2_1 (Multiplexor de Escritura):

Para la implementación del módulo MUX2_1, ya teníamos una referencia de uno 5 a 1 realizado anteriormente, por lo que solo se modificó para que sea 2 a 1 ya que observamos que cuenta con 3 entradas (dos de datos y una de control) y 1 salida. Este módulo se encarga de tomar la decisión final del flujo de datos, seleccionando si la información que viajará de regreso al Banco de Registros será el resultado matemático calculado por la ALU o el dato recién extraído de la Memoria de Datos.

CONCLUSION

El trabajar con este tipo de problemas constantemente nos permite a través de la experiencia adquirida y la investigación, comprender a niveles más profundos como funciona cada módulo y a su vez en conjunto todo el datapath de tipo R en este caso.

Consideramos que es de alto valor el aprendizaje aquí adquirido, sobre todo observando la proyección a futuro con la implementación del algoritmo bubble sort. Es importante observar el set de instrucciones propuesto en las tablas, para de esta manera ir orientando la solución del problema desde esta fase temprana, pues adquirimos la noción de lo necesario para las fases que vienen.

REFERENCIAS

Britton, R. (2003). *MIPS Assembly Language Programming*. Pearson Education. https://url-del-sitio.com/MIPS_Assembly_Language_Programming_(2003)-libre.pdf

- Patterson, D. A., & Hennessy, J. L. (2014). *Computer Organization and Design: The Hardware/Software Interface* (5th ed.). Morgan Kaufmann.

- Harris, D., & Harris, S. (2012). *Digital Design and Computer Architecture* (2nd ed.). Morgan Kaufmann.